

Assignment1 Report

24B1002 and 24B0902

1 Question 1. SAT-Based Sudoku Solver

Introduction

Sudoku is a popular constraint satisfaction puzzle consisting of a $n \times n$ grid divided into $n^{1/2} \times n^{1/2}$ subgrids. The objective is to fill the grid such that:

1. Each row contains all digits 1 to n exactly once.
2. Each column contains all digits 1 to n exactly once.
3. Each $n^{1/2} \times n^{1/2}$ subgrid contains all digits 1 to n exactly once.

In this project, we model Sudoku as a **Boolean Satisfiability Problem (SAT)** and solve it using the **PySAT** library. The rules of Sudoku are encoded as propositional formulas in Conjunctive Normal Form (CNF), which are then solved using a SAT solver.

Approach Overview

The Sudoku puzzle is reduced to SAT by encoding each possible assignment of a digit to a cell as a propositional variable. The Sudoku rules are translated into CNF clauses that enforce the following constraints:

- Each cell contains exactly one digit.
- Each digit appears exactly once per row.
- Each digit appears exactly once per column.
- Each digit appears exactly once per 3×3 subgrid.

The given clues are encoded as unit clauses, ensuring the solver respects the initial puzzle configuration. After solving, the satisfying assignment is decoded back into a completed Sudoku grid.

Variable Encoding

We represent each possible placement of a digit d ($1 \leq d \leq 9$) in a cell (i, j) ($1 \leq i, j \leq 9$) with a propositional variable:

$$v(i, j, d) = (i - 1) \cdot n^2 + (j - 1) \cdot n + d$$

This is done using the function `cell(i,j,d)`. where $n = 9$ for standard Sudoku. Thus, we have $9 \times 9 \times 9 = 729$ variables.

CNF Constraints

The encoding ensures the following Sudoku rules are satisfied:

a) Cell Constraints

- At least one digit per cell:

$$\forall i, j : (v(i, j, 1) \vee v(i, j, 2) \vee \dots \vee v(i, j, 9))$$

- At most one digit per cell:

$$\forall i, j, d_1 \neq d_2 : (\neg v(i, j, d_1) \vee \neg v(i, j, d_2))$$

This is done by looping over rows and columns and using `valueofcell()` function for each cell.

b) Row Constraints

Each digit d must appear exactly once per row:

$$\forall i, d : (v(i, 1, d) \vee v(i, 2, d) \vee \dots \vee v(i, 9, d))$$

and no digit can appear twice in the same row:

$$\forall i, d, j_1 \neq j_2 : (\neg v(i, j_1, d) \vee \neg v(i, j_2, d))$$

This is done by looping on values, then all columns and using `valueofcell()` function for creating a clause for each row.

c) Column Constraints

Each digit d must appear exactly once per column:

$$\forall j, d : (v(1, j, d) \vee v(2, j, d) \vee \dots \vee v(9, j, d))$$

and no digit can appear twice in the same column:

$$\forall j, d, i_1 \neq i_2 : (\neg v(i_1, j, d) \vee \neg v(i_2, j, d))$$

This is done by looping on values, then all rows and using `valueofcell()` function for creating a clause for each row.

d) Subgrid Constraints

For each 3×3 subgrid starting at (a, b) , each digit d must appear exactly once:

$$\forall d, a \in \{0, 3, 6\}, b \in \{0, 3, 6\} : \bigvee_{i=a+1}^{a+3} \bigvee_{j=b+1}^{b+3} v(i, j, d)$$

and at most once:

$$\forall d, (i_1, j_1) \neq (i_2, j_2) \text{ within the same box} : (\neg v(i_1, j_1, d) \vee \neg v(i_2, j_2, d))$$

To implement this we iterate over each subgrid (block) of a Sudoku of size $n \times n$ and, for each possible digit d , collects all the cells within that subgrid. Then call `valueofcell(cells)` to enforce constraints that ensure the digit d appears exactly once in that subgrid.

Integration with PySAT

1. **Construct CNF:** Generate clauses using the above encoding.
2. **Add clauses:** For any non-zero cell (i, j) with known digit d , append the unit clause:

$$v(i, j, d)$$

3. **Solve:** Use `Solver(name='glucose3')` from PySAT to find a satisfying assignment.
4. **Extract model:** Interpret positive variables from the SAT model to reconstruct the solved Sudoku grid.

Constructing solution from SAT solver output

1. First we initialize an empty `solution` matrix of size $n \times n$ filled with zeros to represent an empty Sudoku grid.
2. Next iterate over the `model` (a list of encoded variables from a SAT solver) and decode each positive value into its corresponding row i , column j , and digit d using integer division and modulo operations.
3. Finally, fill the `solution` matrix at position $(i - 1, j - 1)$ with the digit d , reconstructing the completed Sudoku grid from the SAT model.

Conclusion

We successfully implemented a Sudoku solver using propositional logic and a SAT solver from the PySAT library. By encoding Sudoku rules into CNF, we leveraged SAT-solving techniques to efficiently and correctly solve any standard 9×9 Sudoku puzzle.

2 Question 2. Grading Assignments Gone Wrong

2.1 Introduction

Input: A grid consisting of empty cells, walls, player, boxes and goals; and T (maximum number of steps allowed). The task is to determine whether there exists a sequence of at most T moves that places every box on a goal cell. If a solution exists we return the sequence of moves; otherwise we report “UNSAT”.

We encode the problem as a SAT instance in Conjunctive Normal Form (CNF) and use PySAT to solve it. The SAT encoding uses boolean variables to represent the positions of the player and each box across time steps $0..T$ and adds clauses that capture the rules of Sokoban.

2.2 Grid and notation

We assume a rectangular grid of size $N \times M$. Cells are identified by coordinates (x, y) with $0 \leq x < N$ and $0 \leq y < M$. The grid characters are:

- "P": player start
- "B": box start
- "G": goal cell
- "#": wall
- ".": empty cell

Let DIRS be the set of 4 cardinal directions:

$$\text{DIRS} = \{U : (-1, 0), D : (1, 0), L : (0, -1), R : (0, 1)\}.$$

3 Variable encoding scheme

This section explains exactly what each SAT variable represents and how we compute unique integer IDs for them.

3.1 Player variables

We introduce a Boolean variable for each triple (x, y, t) meaning “the player is at cell (x, y) at time t (or step t)”. The encoder implements this via the function

$$\text{player_id}(x, y, t) = 1 + t * N * M + x * M + y$$

Explanation:

- There are $N * M$ cells in the grid. At each step t , we need a unique ID for each of these cells. So step 0 uses IDs $1..N * M$, step 1 uses IDs $N * M + 1..2N * M$, and so on. This way the same cell at different steps does not get mixed up.
- The formula $x * M + y$ is just a way to give each cell (x, y) inside the grid a unique number. It’s like numbering the cells row by row from left to right. For example, $(0, 0)$ is 0, $(0, 1)$ is 1, $\dots$ $(1, 0)$ is M , etc.
- We add 1 to ensure variable IDs start from 1 (SAT solvers expect positive integer labels).

3.2 Box variables

For each box index $b \in \{0, \dots, k - 1\}$ (where k is number of boxes), and for each (x, y, t) we introduce a Boolean variable meaning “box b is at (x, y) at time t ”. These variables are allocated after all player variables so ranges of player and box IDs do not collide.

The encoder implements this as:

$$\text{box_id}(b, x, y, t) = \text{player_ids} + 1 + t * k * N * M + b * N * M + x * M + y$$

where $\text{player_ids} = (T + 1) * N * M$ reserves the whole block of player variables for all timesteps.

3.3 Why this type of encoding (linearization)

This linear arithmetic mapping is easy to compute, invertible (so decoding is straightforward), and guarantees that every $(entity, position, time)$ maps to a unique positive integer variable as required by SAT.

4 CNF constraints (how game rules are translated)

We list the CNF clauses grouped by logical intent. Each clause is either a disjunction of positive/negative variables (following PySAT convention: positive integers for variables, negative for their negation).

4.1 1. Initial conditions

- The player is at the given start cell at time 0: $player_id(px, py, 0)$ must be true. This is added as a unit clause.
- For each box b , the variable representing its start cell at time 0 is set to true via unit clauses $box_id(b, bx, by, 0)$.

4.2 2. Exactly-one constraints (player and boxes)

We need to ensure that at any time each entity occupies exactly one free cell.

At-least-one For every time t we add a clause that is the disjunction of all player variables for cells that are not walls:

$$player_id(x_1, y_1, t) \vee player_id(x_2, y_2, t) \vee \dots$$

This enforces that the player is somewhere at time t .

At-most-one We enforce at-most-one with pairwise mutual exclusion: for every pair of distinct cells (a, b) we add clause

$$\neg player_id(a, t) \vee \neg player_id(b, t)$$

4.3 3. No player-box overlap

For every timestep and cell we add clauses of the form

$$\neg player_id(x, y, t) \vee \neg box_id(b, x, y, t)$$

for all boxes b . This prevents a box and the player from occupying the same cell simultaneously.

4.4 4. Player movement (local transitions)

We encode that the player can only move to adjacent, non-wall cells between t and $t + 1$.

For each cell (x, y) at time t (that is not a wall) we collect the set of valid adjacent cells $N(x, y)$ (subset of the four DIRS that remain in bounds and are not walls). We add:

1. A clause $\neg player_id(x, y, t) \vee \bigvee (x', y') \in N(x, y) player_id(x', y', t+1)$ enforcing that if the player is at (x, y) at time t , he must move to one of the adjacent valid cells at time $t + 1$ (this is written as an implication $player(x, y, t) \rightarrow \bigvee player(x', y', t + 1)$).
2. Pairwise exclusion clauses among the $player_id(x', y', t + 1)$ to ensure the player picks at most one adjacent cell as the destination.

This pair of encodings together enforces "player moves to exactly one adjacent free cell".

4.5 5. Box push rules (synchronous pushes)

Pushing is the critical transition: if the player moves into a cell occupied by a box, the box must be pushed into the next cell in the same direction (and that next cell must be free and in-bounds).

For a triple of cells aligned as $P \rightarrow B \rightarrow C$ (player at $P = (x, y)$ at time t , box at $B = (x + dx, y + dy)$ at time t , and cell $C = B + (dx, dy)$ is in-bounds and not a wall) we add clauses that express:

- If the player is at P at t and moves to B at $t + 1$ while there is a box at B at t , then the box must be at C at $t + 1$.

Concretely the following clauses are added (for each box index b and direction):

$$\begin{aligned} &\neg player(P, t) \vee \neg box_b(B, t) \vee player(B, t + 1) \\ &\neg player(P, t) \vee \neg box_b(B, t) \vee box_b(C, t + 1) \\ &\neg player(P, t) \vee \neg box_b(B, t) \vee \neg player(B, t + 1) \vee box_b(C, t + 1) \end{aligned}$$

The first two clauses force $player(B, t+1)$ and $box_b(C, t + 1)$ to be true if $player(P, t)$ and $box_b(B, t)$ are true. The third clause is a safety clause that helps block illegal moves when a push cannot succeed.

4.6 6. Box persistence (unless pushed)

If a box is at (x, y) at time t and is not pushed from any neighboring cell into a different cell at time $t + 1$, then it must remain at (x, y) at time $t + 1$. We implement this by enumerating all possible pushes that would move this box away; if none of those push conditions occur then the box must persist.

Formally, for box b at (x, y) we compute all pairs $(player_pos, box_dest)$ that correspond to pushes moving b from (x, y) to some (x', y') . For each such push candidate we add

$$\neg box_b(x, y, t) \vee \neg player(pos, t) \vee box_b(dest, t + 1)$$

and finally we add a clause

$$\neg box_b(x, y, t) \vee (\bigvee_{pushes} player(pos, t)) \vee box_b(x, y, t + 1)$$

which enforces that if the box is present and no push occurs then it persists.

4.7 7. Goal condition

At the final time T we require each box to occupy some goal cell. For each box b we add the clause

$$\bigvee_{(g_x, g_y) \in Goals} box_id(b, g_x, g_y, T)$$

This forces each box to end up on at least one goal cell; combining this with the exact-one constraints ensures boxes are on goals exactly.

5 Decoding a model into moves

When the solver returns a satisfying assignment (a list of integers, with positive numbers indicating assigned true variables), we reconstruct a move sequence as follows:

1. Convert the list into a set for efficient lookup.
2. For each timestep $t = 0 \dots T-1$, find the unique cell (x_t, y_t) such that $player_id(x_t, y_t, t)$ is true and the unique cell (x_{t+1}, y_{t+1}) such that $player_id(x_{t+1}, y_{t+1}, t+1)$ is true.
3. Compute the difference $(dx, dy) = (x_{t+1} - x_t, y_{t+1} - y_t)$; map it back to a direction symbol U, D, L, R . If any timestep lacks a unique player position the decoding fails.

The decoded move sequence is the sequence of direction symbols found for $t = 0 \dots T-1$.

1. Note this method assumes the encoding enforces exactly-one player position per time layer, so decoding is straightforward.

Contributions

This project was collaboratively completed by a team of two members. Both members contributed equally to the design, implementation, testing, and documentation for both questions. The contributions are summarized below:

Question 1: SAT-Based Sudoku Solver

- **24b1002**

- Designed the variable encoding of Sudoku constraints for CNF, including cell, row, column,
- Report for SAT-Based Sudoku Solver

- **24b0902**

- Converting the satisfying assignment given by the SAT solver to solution
- Included subgrid constraints

Question 2: SAT-Based Sokoban Solver

- **24b0902**

- Developed the encoding for player and box variables across spatial and temporal dimensions.
- Encoded movement, pushing constraints, and goal state conditions into CNF.
- Report of SAT-Based Sokoban Solver

- **24b1002**

- Implemented the linearization of multi-dimensional variables into unique SAT solver IDs.
- Integrated CNF with `PySAT`, extracted the move sequence, and optimized solver performance.
- Report of SAT-Based Sokoban Solver

Both members jointly:

- Brainstormed approaches, derived logical constraints, and cross-verified CNF correctness.
- Conducted testing, debugging, and validation of results for both problems.
- Collaboratively prepared the final report, ensuring detailed explanations and clarity.